

Simple Bank Application Project

1. Objective

Build a simple banking system where users can:

- Create an account
- View account details
- Deposit money
- Withdraw money
- View transaction history

This project helps students understand: - MVC architecture - REST APIs - Database design (SQL) - Basic frontend-backend integration

2. Tech Stack (Suggested)

- Backend: Node/ Spring Boot (Java)/ Python
 - Frontend: React with basic HTML, CSS, JavaScript
 - Database: MySQL
 - Tools: VS Code, Postman, Swagger
-

3. High-Level Architecture

Frontend (UI) → REST API (Controller) → Service Layer → Repository → Database

4. Database Design

4.1 Tables

USERS

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

ACCOUNTS

```
CREATE TABLE accounts (  
    account_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    balance DECIMAL(10,2) DEFAULT 0,  
    account_type VARCHAR(50),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

TRANSACTIONS

```
CREATE TABLE transactions (  
    txn_id INT PRIMARY KEY AUTO_INCREMENT,  
    account_id INT,  
    txn_type VARCHAR(20),  
    amount DECIMAL(10,2),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)  
);
```

5. Backend Design (MVC)

5.1 Model (Entities)

- User
- Account
- Transaction

5.2 Repository Layer

- UserRepository
- AccountRepository
- TransactionRepository

5.3 Service Layer (Business Logic)

AccountService Methods:

- createAccount(userId, accountType)
 - getAccount(accountId)
 - deposit(accountId, amount)
 - withdraw(accountId, amount)
 - getTransactions(accountId)
-

5.4 Controller (REST APIs)

Create Account

POST /api/accounts

```
{  
  "userId": 1,  
  "accountType": "SAVINGS"  
}
```

Get Account Details

GET /api/accounts/{id}

Deposit Money

POST /api/accounts/{id}/deposit

```
{  
  "amount": 500  
}
```

Withdraw Money

POST /api/accounts/{id}/withdraw

```
{  
  "amount": 200  
}
```

Transaction History

GET /api/accounts/{id}/transactions

6. Business Rules

- Cannot withdraw more than balance
 - Deposit amount must be positive
 - Maintain transaction record for each deposit/withdraw
-

7. Frontend (Basic UI Screens)

7.1 Home Page

- Buttons:
 - Create Account
 - View Account
-

7.2 Create Account Page

Fields: - Name - Email - Account Type (Dropdown)

Button: Submit

7.3 Account Details Page

Display: - Account ID - User Name - Balance

Buttons: - Deposit - Withdraw - View Transactions

7.4 Deposit Page

- Input: Amount
 - Button: Submit
-

7.5 Withdraw Page

- Input: Amount
 - Button: Submit
-

7.6 Transaction History Page

Table: - Transaction ID - Type (Deposit/Withdraw) - Amount - Date

8. Sample JSON Responses

Account Response

```
{
  "accountId": 1,
  "userName": "John Doe",
  "balance": 1000.00
}
```

Transaction Response

```
[
  {
    "type": "DEPOSIT",
    "amount": 500,
    "date": "2026-03-20"
  }
]
```

9. Bonus Enhancements (Optional)

- Add login/authentication
 - Add transfer between accounts
 - Add pagination for transactions
 - Add validation messages on UI
-

10. Evaluation Criteria

- Correct API implementation
 - Clean MVC separation
 - Working UI flows
 - Proper database usage
-

11. Submission Requirements

- Source code (GitHub)
 - SQL script
 - Screenshots of UI
 - Postman collection
-

This project is intentionally simple and designed for beginners to understand full-stack development basics.